

USER GUIDE RU — Audion Hub Manager

Это пользовательское руководство по запуску, настройке, MIRROR, Git/Auth, Editor/Diff и восстановлению проекта.

1. Запуск

Из корня проекта:

```
launcher_gui.cmd
```

CLI smoke:

```
launcher_cli.cmd --mirror-preview demo_local --json  
launcher_cli.cmd --mirror-apply demo_local --json  
launcher_cli.cmd --mirror-apply demo_local --apply --json
```

Если portable runtime отсутствует:

```
install\Build_Portable_Env.cmd
```

2. Первичная настройка

Откройте:

```
config/projects.json
```

Для каждого проекта задайте:

- `source_path` — живой проект; относительные значения считаются от корня Hub Manager.

- `projection_path` — Hub Data зеркало.
- `docs_path` — необязательный Markdown/text слой для чтения.
- `profile` — профиль MIRROR из `config/projection_profiles.json`.

Для portable-проектов внутри дерева менеджера лучше использовать относительные пути:

```
{
  "id": "audion_hub_manager",
  "title": "Audion Hub Manager",
  "source_path": ".",
  "projection_path": "S:/Audion/Hub Data/Audion Hub Manager",
  "docs_path": "S:/Audion/Docs/Projects/Audion Hub Manager",
  "profile": "audion_python_project_projection",
  "default_branch": "main"
}
```

Folder picker и scanner сохраняют относительный путь, если выбранная папка находится внутри дерева Hub Manager. Абсолютные пути остаются рабочим fallback для внешних Hub Data и Docs расположений.

Source не должен меняться MIRROR-движком. Hub Data может пересоздаваться.

3. Выбор проекта и скан папки с проектами

Dropdown `Проект` выбирает одну запись из `config/projects.json`. При выборе меняется вся связка:

Project/Source -> Hub Data/Mirror -> Docs

Поэтому рабочая схема для папки с 20 проектами такая:

1. Нажмите `Rebuild dropdown` в `SOURCE ACTIONS` или `Скан проектов` в Storage.
2. Выберите общую папку, например `S:/TOOLS/Apps/`.
3. Hub Manager найдёт реальные корни проектов, включая двойные вложения `Project/Project`.
4. Найденные проекты появятся в dropdown `Проект`.
5. Переключайте проекты по одному и запускайте MIRROR/commit для активного.

Панель `Структура` содержит три переключателя слоя дерева:

```
PROJECT -> source_path
GIT COPY -> projection_path
DOCS     -> docs_path
```

Компактный badge рядом с заголовком **Структура** показывает выбранный слой и разрешённый путь. Иконки папок выбирают новое расположение для слоя, а кнопка очистки удаляет overrides. Search в дереве только фильтрует файлы внутри уже выбранного слоя; проект он не переключает.

Сканер не меняет Source-папки. Он пишет только недостающие записи в

config/projects.json.

Если после переносов или старых seed-конфигов в dropdown остались битые записи, нажмите

Support -> Clean projects.json. Очистка удаляет только записи с отсутствующим

source_path и дубликаты из **config/projects.json**; реальные папки проектов не трогаются.

SOURCE -бейдж под заголовком **SOURCE ACTIONS** обновляется после **Batch Git status** и показывает общую сводку по реестру:

```
SOURCE: <projects> PROJECTS / <clean> CLEAN / <dirty> DIRTY / <errors> ERRORS
```

4. MIRROR preview и apply

Сначала посмотрите план:

```
launcher_cli.cmd --mirror-preview <project_id> --json
```

Потом применяйте осознанно:

```
launcher_cli.cmd --mirror-apply <project_id> --apply --json
```

MIRROR копирует только разрешённые файлы. Runtime, кеши, логи, сборки, бинарные payload-файлы, локальные override-файлы и секреты должны остаться вне Hub.

5. Рабочие окна GUI

Левая часть приложения:

Открыть

Open Project | Mirror | Docs Folder
Open in VS Code | Terminal | Git

MIRROR

Dry-run | Exact mirror | .gitkeep dirs | BLAKE3 compare
Preview MIRROR | Apply MIRROR | Refresh

SOURCE ACTIONS

Rebuild dropdown | Clone Source | Batch Preview MIRROR
Batch Safety Scan | Batch Verify Mirror | Batch Git status
Combined workspace

PROJECT ACTIONS

Refresh tree | Load selected to Editor | Open in VS Code
Copy relative path | Copy full path

Support

Auth Doctor | BLAKE3 | Verify Mirror | Storage | Safety Scan
Clean projects.json

SOURCE ACTIONS работают со всем реестром проектов. **PROJECT ACTIONS** работают с текущим выбранным объектом дерева.

Terminal открывает внешний терминал в активном Source-проекте. **Git** открывает внешний терминал в текущем Git-root и сразу выполняет `git status --short`, поэтому это Git-инспектор, а не ещё одна кнопка открытия папки.

Правая часть приложения:

Quick | Branch | Editor | Diff | Storage
Remote | Basket | Reader | History | Details

Quick — частые локальные Git/file команды. Нижняя command area — ручной textarea для команды: выбор команды из cache/pinned копирует её туда, а run выполняет ровно этот текст.

Вместе Git-панели закрывают почти весь повседневный lifecycle: создание репозитория, status, root lookup, history, diff/blame/show, staging, restore, commit, tags, remotes, branch switching/creation, stash, revert, merge, cherry-pick, bundle, clean preview, gc, fsck, config diagnostics, CI/CD observation и clone into Source. Branch switching, integration, stash и branch-danger templates живут в **Branch**, а не в **Quick**, чтобы рискованные веточные операции оставались в одном

workflow-окне. Оставшийся зазор намеренный: редкие разрушительные сценарии остаются видимыми command templates, а не магией в один клик.

Блок **GIT LOCAL** — первая точка для повседневного состояния репозитория: **init** создаёт видимый **.git**, **status** показывает текущие изменения, **root** выводит корень репозитория, **log --oneline** даёт короткую историю, **reflog** помогает найти точки отката. **user config** показывает **user.name**/**user.email** с источником в Auth-блоке внутри **Remote**. **config list** выводит весь Git config с файлами-источниками в Git backup/maintenance. Graph-история остаётся во вкладке **History**.

Basket — подготовка читаемого коммита: type, scope, subject, message, version/tag.

Branch — branch status, **branch -vv**, **switch**, **switch -c**, tags, compare branches, **merge --no-ff**, **revert**, **cherry-pick**, stash и abort/rebase templates.

Remote — remotes, push/pull/fetch и настройка авторизации. Форма remote собирает SSH URL из платформы, логина/группы и имени репозитория, сохраняет enabled remotes в **config/remotes.json** и держит recent values в **config/remote_field_cache.json**. Значения из recent-select подставляются только отдельной кнопкой use, без автоматического наложения на ручной ввод.

Частые remote-команды идут первыми:

```
push origin | pull --ff-only
fetch --all --prune | remote -v
push all remotes | apply remotes.json
origin push URLs | собрать URL
сохранить remote
```

push origin кладёт в очередь **git push --follow-tags origin <branch>**, чтобы annotated checkpoint tags тоже публиковались. **push all remotes** пушит ветку и tags во все remotes через Python-логику, без PowerShell-specific shell loop.

Editor — быстрый редактор Markdown/text:

```
.md
.markdown
.txt
.rst
```

Toolbar редактора icon-only с tooltip: load selected, save, paste из Windows clipboard, copy, clear, открыть текущий файл в VS Code и expand/collapse окна.

Diff — RedLine-представление изменений.

Auth-действия живут внутри **Remote**: **Check Auth**, парные GitHub/GitLab SSH probes, парные **gh auth login** / **glab auth login**, Windows Credential Manager, GitKraken folder и VS Code. Login открывает внешний терминал один раз; Hub Manager не хранит токены или пароли.

Storage — корни Manager/Hub Data/Docs, проверка раскладки, Safety Scan output и machine-local picker/test/save для **Code.exe**. Эти значения хранятся в **config/apps.local.json**, а не в общем project config.

6. BLAKE3 и Verify Mirror

BLAKE3 — быстрая диагностика backend: она проверяет, что MIRROR использует настоящий BLAKE3, а не SHA-256 fallback.

Verify Mirror — read-only сверка Source ↔ Hub Data после зеркалирования. Она строит BLAKE3-манифесты в памяти, сравнивает **same / changed / missing / extra** и пишет один датированный JSON в:

```
logs/<project_id>/
```

Source, Hub Data и Docs при этом не получают manifest-файлов. Docs/Obsidian/ LogSeq/VS Code docs-папки отдельно не хэшируются: они читающий слой, а каноническая техническая копия уже находится в Hub Data.

7. Git workflow

Hub Manager работает с обычными **.git**:

```
Full Project Source/.git  
Hub Data/<project>/ .git
```

Коммиты, созданные Hub Manager, должны stage только pathset активного Hub profile. Не используйте широкое **git add .** для таких checkpoint-коммитов.

Хорошее имя коммита:

```
docs(hub): projection v0.1.17 - refresh builder menu
```

Версия задаётся вручную. Это намеренно: автоматическая версия слишком легко создаёт шум и ложную важность.

8. Remote и Auth

Hub Manager не хранит токены.

Используйте:

```
gh auth login
glab auth login
ssh -T git@github.com
ssh -T git@gitlab.com
```

Windows Credential Manager, VS Code и GitKraken можно использовать для логина, проверки credential helper и визуальной работы с конфликтами.

После одноразовой авторизации на машине обычная синхронизация такая:

```
машина A: commit -> tag -> push origin
машина B: fetch --all --prune -> pull --ff-only
```

`fetch` обновляет remote tracking information без изменения локальных файлов. `pull --ff-only` используется, когда нужно осознанно продвинуть текущую ветку.

8.1. Свой сервер Forgejo или Gitea

Верхний ряд окна **Remote** — кнопки платформ: **GitHub**, **GitLab**, **Codeberg**, **Forgejo**, **Gitea**, **Свой сервер**. Три последние дополнительно показывают поля **Адрес сервера** и **SSH-порт**; облачным платформам они не нужны. Рядом второй ряд кнопок — **SSH-ключ** или **HTTPS-токен**; он определяет, какой вид URL соберёт кнопка **собрать URL**.

```
порт 22          -> git@host:owner/repo.git
другой SSH-порт -> ssh://git@host:PORT/owner/repo.git
HTTPS            -> https://host/owner/repo.git
```

Известные серверы перечисляются в `config/forgejo_hosts.json`. Там нет и не должно быть токенов: только адрес, SSH-порт, предпочитаемый вид URL и логин, который подставляется после первой проверки.

Порядок подключения:

1. `проверить сервер` — спрашивает у сервера версию API. Проверяет адрес до того, как в дело идёт токен.
2. `страница токенов` — открывает в браузере `Settings -> Applications` вашего сервера. Там создайте токен и выдайте ему права:

Что делаете	Какое право
«проверить сервер»	токен не нужен
«кто я»	<code>read:user</code>
«список репозитория»	<code>read:user</code>
«создать репозиторий»	<code>write:user</code>
<code>git clone</code> / <code>git push</code> по HTTPS	<code>read:repository</code> / <code>write:repository</code>

Права считаются **по маршруту, а не по объекту**. Создание репозитория живёт по адресу `/api/v1/user/*`, поэтому требует `write:user`, а вовсе не `write:repository` — с последним сервер отвечает `403 ... required scope(s): [write:user]`. И наоборот: токен с одними лишь `read:user, write:user` проходит все кнопки в окне, но `git clone` по HTTPS отваливается с `remote: Forbidden` — git проверяется отдельно.

Проще всего выдать токenu и user-, и repository-права сразу. Forgejo при сохранении схлопывает лишнее: `read:user, write:user, write:repository` ложится в базу как `write:repository, write:user`, потому что write включает read. Всё проверено опытом на живом Forgejo 15, отдельным токеном на каждую операцию. Hub Manager показывает нехватку права отдельным сообщением — «токен верный, но создан без нужного права» — и называет то, которое просит сервер.

3. Вставьте токен в поле `Токен доступа`. Этого уже достаточно — кнопки `кто я`, `список репозитория` и `создать репозиторий` работают сразу с тем, что введено. Кнопка `запомнить токен` нужна для другого: она проверяет токен запросом `/api/v1/user` и отдаёт его вашему Git credential helper, чтобы он пережил перезапуск программы и чтобы им мог пользоваться сам `git`. В файлы проекта токен не попадает ни в том, ни в другом случае.

Для `git clone` и `git push` по HTTPS запомнить токен **обязательно**: git читает учётные данные из хранилища и поля в окне не видит.

4. `кто я`, `список репозитория`, `взять репозиторий` — работают с уже сохранённым токеном. `взять репозиторий` заполняет логин и репозиторий из выбранной строки и сразу собирает remote URL.
5. `создать репозиторий` создаёт репозиторий под вашей учётной записью из поля `Репозиторий`; видимость задаётся кнопками `приватный` / `публичный`.
6. Иконка с перечёркнутым ключом рядом с полем токена забывает его: очищает поле и удаляет запомненную копию из хранилища учётных данных. На сервере токен продолжает действовать, пока вы не отзовёте его там же, на странице токенов.

Отправка кода аутентифицируется отдельно и обычным способом: либо SSH-ключом (проверка — кнопка `ssh -T Forgejo`, она учитывает нестандартный порт), либо тем же токеном по HTTPS, который `git push` берёт из хранилища сам.

`Check Auth` показывает по каждому серверу: доступен ли он, сохранён ли токен, действителен ли он ещё и какие credential helper настроены.

8.2. Большой первый push через прокси или туннель

Если инстанс published через реверс-прокси или туннель с ограничением размера тела запроса, первый push репозитория может вернуть `413`. Git отправляет пачку объектов одним запросом, поэтому лимит бьёт именно по начальной загрузке истории, а не по последующим мелким коммитам. У Cloudflare Tunnel на бесплатном тарифе это 100 МБ, и для туннеля лимит не обходится настройками DNS.

Обход — отправить код по SSH мимо HTTP-прокси. Если SSH-порт слушает только localhost на сервере, пробросьте его на время push:

```
ssh -N -L 2222:127.0.0.1:2222 user@server
git push ssh://git@127.0.0.1:2222/owner/repo.git
```

Hub Manager собирает такие URL сам: выберите `SSH-ключ`, укажите адрес и SSH-порт, и кнопка `собрать URL` даст форму `ssh://git@host:PORT/owner/repo.git`. Git LFS проблему не решает, если он выключен на сервере или ходит через тот же прокси.

9. VS Code

Путь к `Code.exe` зависит от машины. Общий config не должен хранить такие пути.

Machine-local путь храните в:

```
config/apps.local.json
```

Файлы `*.local.json` не должны попадать в Hub Projection и Git-коммиты.

10. Восстановление из Hub Data

Если Source удалён, но Hub Data жив:

```
git -C "<Hub Data>\Audion Hub Manager" log --oneline -n 5
robocopy "<Hub Data>\Audion Hub Manager" "<portable-root>\Audion Hub Manager" /E
/COPY:DAT /DCOPY:DAT /R:1 /W:1 /XJ
```

После восстановления пересоберите runtime и PDF/release артефакты, если они нужны.

11. PDF

PDF не является источником истины. Источник — Markdown.

PDF можно пересоздать внешним движком:

```
python "E:\TOOLS\Audion Office OCR AI\system_core\dev_markdown_pdf_engine.py"
```

12. Минимальная проверка

```
python -m compileall -q system_core
python -m pytest -q tests
python system_core\ui_nicegui\app.py --smoke
```

Ожидаемый baseline: `59 passed`.